

Allora Implementation Plan

Version 1.0 (Draft) | Based on: Allora Product Specification v1.0 | Audience: development team

1. Purpose

This plan explains how to build Allora in the order set by the specification. It lists the technology choices, the work packages for each release phase, the requirement IDs each package covers, and the checks that show a package is finished. Requirement IDs (for example ACC1 or FIN5) match the specification, so every task can be traced back to it.

Estimates assume one experienced full stack developer. With two developers (one on the frontend and one on the backend), expect about 30 to 40 percent less calendar time. All estimates are rough and should be reviewed at the end of Phase 0.

2. Approach in Brief

- Build in the four phases of the release plan. Each phase ends with a usable release that can be tested with real users.
- Build the shared foundation first: accounts, data layer, calendar, reminders, notifications, and the app shell. Every module depends on these.
- Build each module as a vertical slice. Database tables, API, user interface, and tests are delivered together.
- Build the items shared by many modules once, in a general way: calendar events, reminders, notes, attachments, tags, and goal links.
- Turning a module on or off (ACC5) hides it in the interface. The data stays in the database.
- Tie every record to its owner and store the created and changed details the specification requires.

3. Timeline Overview

- Phase 0, Project setup: 1 to 2 weeks
- Phase 1, Foundation: 10 to 12 weeks
- Phase 2, Money and projects: 8 to 10 weeks

- Phase 3, Research and insights: 6 to 8 weeks
- Phase 4, Growth: 8 to 10 weeks
- Total: about 33 to 42 weeks for one developer

4. Technology Decisions

The specification proposes the stack below. Items marked as a suggestion are not in the specification and can be replaced by the developer.

- Frontend: React with TypeScript and a responsive layout. Suggested additions: a router, a data fetching and caching library (for example TanStack Query), a form library with schema validation, Tailwind CSS for styling, a charting library for reports, a drag and drop library for boards, and a calendar component that supports day, week, month, and agenda views.
- Backend: Node.js REST API organized by module (suggestion: Express or Fastify, with schema based request validation).
- Database: PostgreSQL for all structured data, with a migration tool and a query layer or ORM (suggestion: Prisma or Drizzle).
- File storage: Cloud object storage for receipts, PDFs, and attachments. Use signed upload links, and limit file type and size.
- Authentication: Email and password with secure sessions, plus Google sign in. Hash passwords with a strong algorithm such as argon2 or bcrypt.
- Background jobs: A job queue for reminders, recurring transactions, recurring tasks, email sending, and cleanup of deleted items (suggestion: BullMQ with Redis, or a queue that runs on PostgreSQL).
- Notifications: An email service, browser push notifications through a service worker, and an in app notification list.
- Hosting: A cloud provider with separate test and live environments, HTTPS everywhere, and daily database backups.
- Quality tools: Unit tests, integration tests, and end to end tests (suggestion: Playwright), run automatically on every code change.
- Monitoring: Error tracking, request logging, and an uptime check.

5. Code Structure

Use one repository with three parts:

- apps/web: the React frontend, with one folder per module.
- apps/api: the Node.js API, with one folder per module (routes, services, data access, tests).
- packages/shared: types and validation schemas used by both sides.

Module folders match the specification: accounts, dashboard, habits, finance, academics, tasks, projects, research, goals, calendar, search, insights, settings.

6. Phase 0: Project Setup (1 to 2 weeks)

Tasks:

- Create the repository, folder structure, code style rules, and commit rules.
- Set up the continuous integration pipeline: lint, type check, unit tests, and build on every change.
- Create test and live environments, with secrets kept outside the code.
- Create the database, the migration process, and a seed data script for development.
- Define API conventions (see section 11) and set up generated API documentation.
- Create the design tokens: colors, spacing, type sizes, and one accent color for each module (section 11 of the specification), for light and dark themes.
- Decide the open questions listed in section 16, because several change the data model.
- Design the ownership model so that records can later be shared (needed for PRJ8). Every record has an owner, and access checks go through one shared function.

Done when: a blank app deploys automatically to the test environment, the pipeline passes, and the team agrees on the decisions in section 16.

7. Phase 1: Foundation (10 to 12 weeks)

Goal: a person can sign up, choose modules, and manage habits, academics, tasks, and a shared calendar with reminders. Light and dark themes and data export are included.

7.1 Accounts and authentication (about 1.5 weeks)

Covers: ACC1, ACC2, ACC4, SET5

- Sign up and log in with email and password, and with Google.
- Email verification and password reset by email, with expiring links.

- Sessions that expire after inactivity, and a page that lists active sessions with the option to end them.
- Profile: name, photo, time zone, default currency, language, and first day of the week.
- Password change.
- Protection against repeated login attempts.

Done when: all flows work in the browsers listed in section 12, and security tests for session handling and password storage pass.

7.2 Core data layer (about 1 week)

Covers: Data model (specification section 8), ACC6 in part

- Create common columns for every table: owner, created at, created by, and updated at.
- Soft delete with a deleted at column. A daily job removes records that were deleted more than 30 days ago.
- Create the shared tables for notes, attachments, and tags, linked to other records by type and ID.
- Add a shared access check so a user can only read and change their own records.
- Plan nullable links on tasks to a project, goal, or course. Add the links when those modules are built.

Done when: tests prove that one user cannot read or change another user's records, and the cleanup job removes expired deleted records.

7.3 App shell and design system (about 1.5 weeks)

Covers: specification section 11, SET1

- Navigation: sidebar on desktop and bottom navigation bar on phones.
- Light and dark themes, with the choice saved to the profile.
- A shared set of components: buttons, forms, cards, tables, dialogs, empty pages that explain the module and offer a first action, and friendly error messages.
- The quick add button, available on every page (DSH3). Add actions for modules as they are built.
- Keyboard shortcuts for frequent actions on desktop.
- Accessibility from the start: keyboard use, screen reader labels, and color contrast that meets WCAG 2.1 level AA.

Done when: the shell works on phone, tablet, and desktop sizes, and an automated accessibility check passes.

7.4 Onboarding and module settings (about 1 week)

Covers: ACC3, ACC5, user flow 7.1

- A short setup after sign up: choose modules, enter time zone and currency, and add a first item in each chosen module.
- Store module choices per user, and let users change them later in settings.
- Hide navigation, dashboard widgets, and quick add actions for modules that are off.

Done when: a new user can finish setup and add a first item in under 5 minutes.

7.5 Calendar and reminders engine (about 2 weeks)

Covers: CAL1 to CAL6, CAL7 in part (ICS export), ACA9 in part

- Create a general calendar event model. Other modules create events by linking an item (class, exam, task, milestone, deadline, bill, habit reminder). Events can also be created by the user (CAL4).
- Day, week, month, and agenda views, with a filter by module (CAL2, CAL3).
- Create a reminder model. A reminder belongs to one item and has one or more offsets, such as one week, one day, and two hours before.
- A background job finds reminders that are due and sends them in the app, by email, and as browser notifications, depending on the user's choices (CAL5).
- Quiet hours, applied when sending (CAL6).
- Export the calendar as an ICS file.
- Handle time zones and daylight changes correctly by storing times in UTC along with the user's time zone.

Done when: an item from a test module appears on the calendar, and a reminder arrives on every selected channel at the right time, except during quiet hours.

7.6 Job and task manager (about 2 weeks)

Covers: JOB1 to JOB9

- Tasks with title, description, due date, priority, status, labels, subtasks, and attachments (JOB1).

- Repeating tasks: daily, weekly, and monthly. The background job creates the next task when one is completed or when its date arrives (JOB2).
- List, board, calendar, Today, and Upcoming views (JOB3).
- A context on every task, such as Work or Personal (JOB4), and a place to save job details (JOB5).
- A timer that starts and stops on a task, with total time shown (JOB6).
- Notes that contain action items which turn into tasks with one click (JOB7).
- Filter, sort, and saved filters (JOB8).
- A daily plan view (JOB9).

Done when: tasks appear on the calendar and dashboard, and repeating tasks are created on schedule.

7.7 Academic manager (about 2.5 weeks)

Covers: ACA1 to ACA9

- Terms and courses with name, code, instructor, credit hours, class schedule, and color (ACA1).
- A weekly timetable (ACA2). Class sessions repeat weekly and show on the calendar.
- Exams with type, date, time, room, syllabus topics, and a countdown (ACA3).
- Assignments, projects, and presentations with due date, weight, and status (ACA4).
- Marks for each assessment, the running course grade, and a GPA and CGPA calculator with an editable grading scale (ACA5).
- A target grade planner that shows the marks needed in remaining assessments (ACA6).
- Notes, files, and links for each course (ACA7).
- Attendance tracking with a warning below a chosen level (ACA8).
- Reminders for exams and deadlines at chosen intervals (ACA9), using the reminders engine.
- Keep grade calculations in one tested module, because errors here are easy to notice and costly for users.

Done when: user flow 7.2 (preparing for an exam) works from start to finish, and grade calculations match a set of hand checked examples.

7.8 Habit tracker (about 1.5 weeks)

Covers: HAB1 to HAB7 (HAB8 is added in Phase 2 with Goals)

- Habits with name, icon, color, category, frequency, and reminder time (HAB1, HAB2).
- Yes or no, count, and timed habits (HAB3).
- Mark complete from the dashboard or the habits page, and fill in missed days (HAB4).
- Current and longest streak, calculated from the logs with the user's time zone and chosen frequency (HAB5).
- A calendar heatmap and charts for weekly and monthly completion (HAB6).
- Pause or archive without losing history (HAB7).

Done when: streaks are correct for daily, weekday, and times per week habits, including across time zone changes.

7.9 Dashboard (about 1 week)

Covers: DSH1 to DSH5

- Widgets for tasks due, habits to complete, and upcoming exams and deadlines. Widgets for spending and project progress are added in Phase 2.
- Add, hide, and rearrange widgets, with the layout saved per user (DSH2).
- Daily progress indicator (DSH4) and weekly summary card (DSH5).
- Fetch dashboard data with a small number of requests, so the page loads in under 2 seconds.

Done when: the dashboard meets the load time target with a realistic amount of test data.

7.10 Settings and data export (about 1 week)

Covers: SET2, SET3, SET4, ACC6

- Notification settings: types, timing, and quiet hours (SET2).
- Module on and off settings (SET3).
- Export all user data in JSON, and import it again (SET4, ACC6).
- Delete the account permanently, including files in storage, with a confirmation step (ACC6).

Done when: an exported file can be imported into a new account without loss, and account deletion removes all data.

7.11 Hardening and beta (about 1 week)

- Full end to end test run for the main flows.
- Accessibility review, security review, and performance check.
- A closed beta with a small group of students. Collect feedback and fix serious problems.

Done when: Phase 1 acceptance checks in section 13 pass and the beta group completes the main flows without help.

8. Phase 2: Money and Projects (8 to 10 weeks)

8.1 Finance manager (about 3.5 weeks)

Covers: FIN1 to FIN11

- Accounts with a starting balance (FIN1).
- Income and expenses with amount, date, category, account, note, and an optional receipt photo (FIN2).
- Transfers between a user's own accounts, saved as two linked entries so balances stay correct (FIN3).
- Custom categories and subcategories (FIN4).
- Monthly budgets per category, with warnings at 80 percent and 100 percent (FIN5).
- Recurring transactions created by the background job (FIN6).
- Savings goals with target amount, date, and progress (FIN7).
- Money owed and money owed to the user, with due dates (FIN8).
- Reports: spending by category, income compared with expenses, and monthly trends, built with database queries that group by month and category (FIN9).
- Any currency, with a default currency in settings (FIN10).
- CSV export of transactions (FIN11).
- Store money as whole numbers in the smallest currency unit, together with a currency code. Do not use floating point numbers for money.
- Add recurring bills and debt due dates to the calendar.

Done when: user flow 7.3 (recording an expense) works, balances and budgets update at once, and totals match hand checked examples.

8.2 Project manager (about 3 weeks)

Covers: PRJ1 to PRJ7

- Projects with name, description, goal, start date, end date, status, and color (PRJ1).
- Milestones and tasks inside projects. Project tasks are rows in the same task table, linked to a project, so they also appear in the main task list (PRJ2).
- Progress calculated from completed tasks and milestones, with an option to set it by hand (PRJ3).
- Board, list, and timeline views (PRJ4).
- Notes, files, and links for each project (PRJ5).
- Task dependencies, with a check that blocks circular dependencies (PRJ6).
- Project templates: thesis, event, and website launch (PRJ7).
- Add milestones to the calendar.

Done when: project progress updates when tasks change, and the timeline view handles dependencies correctly.

8.3 Goals and weekly review (about 2 weeks)

Covers: GOL1 to GOL4, HAB8, user flow 7.4

- Goals with description, category, target, and deadline (GOL1, GOL2).
- Links from a goal to habits, tasks, projects, and savings goals. Goal progress is calculated from the linked items (GOL3, HAB8).
- Weekly and monthly review prompts and the weekly review flow (GOL4).

Done when: goal progress changes when linked items change, and the weekly review flow works from start to finish.

8.4 Global search (about 1 week)

Covers: SRC1

- One search box across all modules, using PostgreSQL full text search with an index per searchable table.
- Results grouped by module, always limited to the current user's records.

Done when: search returns results in under 1 second with realistic data.

8.5 Dashboard and calendar updates (about 0.5 week)

- Add the spending and project progress widgets to the dashboard.
- Confirm that bills and milestones appear on the calendar.

9. Phase 3: Research and Insights (6 to 8 weeks)

9.1 Research tracker (about 3 weeks)

Covers: RES1 to RES8

- Research topics with title, main question, status, and notes (RES1).
- A paper library with title, authors, venue, year, link or PDF, tags, reading status, summary, key takeaways, and rating (RES2).
- Notes and highlights linked to papers (RES3).
- An experiment log (RES4).
- Conference and journal deadlines that appear on the calendar (RES5).
- Import a paper by DOI, using a public metadata service, and export references in BibTeX (RES6).
- Plan paper sections with word count targets (RES7) and reading goals (RES8).

Done when: a paper added by DOI has its details filled in, and exported BibTeX opens correctly in common reference tools.

9.2 Insights and reports (about 2 weeks)

Covers: INS1, INS2

- An insights page with weekly and monthly summaries: habit completion, tasks finished, spending, study hours, and project progress.
- Study hours come from timed habits and task timers, so confirm the source with the product owner.
- Download any report as a PDF, created on the server.

Done when: report numbers match the numbers shown in each module.

9.3 Google Calendar sync (about 1.5 weeks)

Covers: CAL7 (sync part)

- Connect a Google account and send Allora events to Google Calendar.

- Start with sending events one way. Two way sync is more work and should be decided before this package starts.

Done when: events created in Allora appear in Google Calendar and stay in step after edits and deletions.

9.4 Study planner (about 1.5 weeks)

Covers: ACA10

- Place study sessions on the calendar before each exam, using the exam date, the syllabus topics, and the time the user has available.
- Start with simple rules that the user can adjust. Do not build anything complex before real users show what they need.

Done when: a user with three exams in one month receives a sensible plan that they can edit.

10. Phase 4: Growth (8 to 10 weeks)

10.1 Collaboration on projects (about 4 weeks)

Covers: PRJ8

- Invite others to a project by email, with member roles and permissions.
- Assign tasks to members, and add comments.
- Change access checks so a record can be reached through ownership or project membership. This relies on the ownership design from Phase 0.
- Notify members about assignments and comments.

Done when: security tests prove that members only see projects they belong to, and removing a member removes their access at once.

10.2 Installable mobile app (about 3 weeks)

- Make the web app installable with a manifest and service worker, and support browser push notifications.
- Add limited offline viewing of recent data.
- A native app is a separate decision (see section 16).

Done when: the app installs on Android and iOS and reminders arrive as push notifications.

10.2b Community templates (about 2 weeks)

- Let users share project and habit templates, and let others copy them.
- Add reporting and review of shared templates. This is not a social network, and no profiles or feeds are built.

Done when: a shared template can be found, copied, and reported.

11. API Conventions

- REST with JSON, under a versioned path such as `/api/v1`, with one set of routes per module.
- Standard list requests support pagination, sorting, and filtering.
- One error format with a code, a plain message, and details about which field failed.
- Every request is checked for login and ownership.
- Quick add and other create actions are safe to repeat, so a poor connection cannot create duplicates.
- Rate limits on login, password reset, and email sending.
- API documentation is generated from the code and kept current.

12. Nonfunctional Requirements in Practice

- Performance: Use database indexes on owner and date columns, pagination on all lists, code splitting in the frontend, and caching of repeated reads. Test main pages against the 2 second target.
- Availability: Set up health checks, automatic restarts, and alerts to reach 99.5 percent.
- Scalability: Keep the API stateless so more instances can be added. Run background jobs as separate workers.
- Security: Hash passwords, use HTTPS, expire sessions, check ownership on every query, validate all input, and protect against common web attacks. Run a security review before each phase release.
- Privacy: Collect only the data that is needed. Provide export and deletion. Keep finance and academic data out of analytics and logs.

- Reliability: Daily database backups, with a restore test every quarter.
- Usability: Check that common actions take no more than three taps or clicks.
- Accessibility: Follow WCAG 2.1 level AA and test with a screen reader.
- Compatibility: Test current versions of Chrome, Edge, Firefox, and Safari, and phone, tablet, and desktop sizes.
- Maintainability: Organize code by module, keep tests with the code, and document each module.
- Data portability: Support CSV, JSON, ICS, and BibTeX exports.

13. Testing Plan

- Unit tests for calculations: grades and GPA, budgets, streaks, goal and project progress, repeating tasks, and reminder times.
- Integration tests for each API route, including ownership checks.
- End to end tests for the main flows: setup, preparing for an exam, recording an expense, and the weekly review.
- Security tests for sessions, passwords, and access to other users' data.
- Performance tests with realistic data, such as two years of daily use.
- Accessibility checks, both automated and manual.
- Acceptance check at the end of each phase: every requirement in the phase is tested against its description in the specification.

14. Release and Deployment

- Every change is deployed to the test environment automatically after the pipeline passes.
- A release to the live environment happens at the end of each phase, after acceptance checks and a short beta.
- Use feature switches so unfinished modules stay hidden in the live app.
- Database migrations run automatically and can be rolled back.
- Keep a short release note for each release.

15. Measuring Success

Add privacy friendly event tracking for the metrics in section 13 of the specification:

- Registered users and weekly active users
- Share of users with three or more modules turned on
- Habit logging rate in the first two weeks
- Share of tasks and assignments finished before the due date
- Users still active after 30 days and after 90 days
- Average page load time
- Satisfaction score from a short in app survey

16. Decisions Needed Before Development

- Will Allora be free, paid, or both? This affects accounts, limits, and billing work.
- Should the first release work offline? This affects the architecture of the frontend.
- Which languages should be supported after English?
- Should Google Calendar sync be one way or two way?
- Should the finance module support shared expenses between friends? This affects the ownership model.
- Should the research tracker store uploaded PDFs, or links only?
- Is a native mobile app planned, or is an installable web app enough?
- Which cloud provider and email service will be used?
- What is the team size? This changes the timeline in section 3.

17. Risks and Mitigations

- The scope is large. Mitigation: deliver in phases, and keep each module small and complete before starting the next.
- Sharing records in Phase 4 can force large changes. Mitigation: design ownership and access checks in Phase 0.
- Reminders are unreliable if jobs fail. Mitigation: retry failed jobs, monitor the queue, and test time zones and daylight changes.
- Money and grade calculations can contain errors. Mitigation: unit tests with hand checked examples, and whole number storage for money.
- Data entry feels like work. Mitigation: quick add, recurring items, and sensible defaults, tested with beta users.

- Sensitive finance data needs strong protection. Mitigation: encryption in transit and at rest, strict access checks, and regular security reviews.
- Notifications may annoy users. Mitigation: user control over type, timing, and quiet hours.